Group Mid-Term Project

Project Objective

The primary objective of this group project is to evaluate your ability to design, build, and rigorously test a systematic trading strategy by leveraging the data-driven models, quantitative techniques, and AI methods discussed throughout the course.

This project will require you to apply the full systematic trading workflow:

- idea generation
- data collection
- validation of the data (use prices adjusted for stock splits and dividends)
- feature engineering (if needed)
- building the strategy
- strategy backtesting
- out-of-sample validation
- performance evaluation

The data validation step is critical: using non-adjusted prices will lead to false signals (e.g., a stock split appearing as a crash) and invalidate your entire backtest.

Project Overview & Core Task

Your group will develop a complete, data-driven trading system. The system's final output for any given asset (or assets) at any given time must be a clear, discrete signal:

- **+1:** Go Long (buy)
- **0:** Stay Neutral / Out of the market (flat)
- **-1:** Go Short (sell)

Your system must algorithmically generate these signals based on an analysis of historical financial data and/or alternative data, as we saw in the labs.

In the backtest, the strategy return for each period equals the asset's return when the signal is +1 (long), zero when the signal is 0 (flat), and minus the asset's return when the signal is -1 (short). State any assumptions.

The trading system must trade at least 2 assets.

Groups

Groups consist of 3-5 students. The teaching team may assign different grades to members of the same group if contributions are clearly unbalanced. The mid-term project is reserved for attending students (attendance ≥ 70%).

Core Requirements & Methodology

You must build your system following the structured process discussed in the lectures.

Data Acquisition

Your strategy can be powered by:

- **Historical Financial Data:** Prices, volume, etc. You can source this from Yahoo Finance (yfinance), by using the Bloomberg Terminal (also via BQL), from investing.com, etc.
- **Alternative Data:** This is highly encouraged. Examples include:
    - Sentiment data from news headlines or social media (e.g., scored with FinBERT).
    - Macroeconomic data (e.g., from Bloomberg ECST function).
    - Fundamental data about stocks.
    - Other novel data sources you identify.

Data frequency must be at least weekly. The sample must be long enough for the In-Sample and Out-of-Sample periods to contain different market regimes (at least 10 years for weekly and at least 4 years for daily data; for intraday data, justify the chosen length).

Strategy Development

You may use any of the approaches studied in the course (technical, statistical arbitrage, AI-based) or develop your own. The trading system must include at least one AI technique or optimization method. Combining multiple strategies into an ensemble (e.g., consensus rules, ML-based, or agent-based) is strongly encouraged.

Portfolio Construction & Execution Rules

- **Execution timing**: Explain clearly when each signal is executed relative to the data used to compute it, making sure no lookahead bias is introduced.
- **Transaction costs:** Including transaction costs in your backtest (e.g., a fixed cost in basis points per trade) is optional. If you include them, describe your assumptions.
- **Position sizing**: Describe how capital is allocated across assets (e.g., equal weight) and whether leverage is used.

The Evaluation Framework

You **must** follow the rigorous evaluation framework discussed in class. Your grade will be heavily based on the *process* of your evaluation, not just the final return.

- **Data Splitting:** Clearly split your data into an **In-Sample (Training)** period and an **Out-of-Sample (Testing)** period. OOS period must be big enough to evaluate the strategy. All model selection and hyperparameter tuning must be done on a validation set carved out of the In-Sample period (or via walk-forward within it). The OOS set must be used only once, for final evaluation. Walk-forward validation is appreciated.
- **Performance Metrics:** Evaluate your strategy with appropriate performance metrics, for both In-Sample and Out-of-Sample periods, and compare them against an appropriate benchmark, justifying your choice. Examples of metrics that can be taken into consideration:
    - Cumulative Return (plotted as an Equity Curve, starting at 100, with the benchmark on the same chart)
    - Annualized Return
    - Annualized Volatility
    - Sharpe Ratio (state the risk-free rate you assume)
    - Maximum Drawdown (MDD)
    - Profit Factor
- **Pitfall Analysis:** Show how your methodology addresses the following pitfalls:
    - Overfitting
    - Lookahead Bias
    - Survivorship Bias
    - Data Snooping
    - Pretrained Model Leakage

Deliverables

You will submit a **single file** `group_<group-number>.zip` (e.g., `group_4.zip`) containing:

- `project/`: The code repository of your project;
- `presentation.pdf` (or `.pptx`): The slides used to present your project

Group numbers will be assigned by the teaching team.

Each group will present its project in a 10-minute presentation (followed by Q&A) on one of the presentation dates (see Deadlines & Presentation Dates).

Project code

You are strongly encouraged to maintain a clean and well-organized repository structure within your submission. You are free to organize your repository as you prefer, as long as:

- the pipeline is implemented in Python scripts (`.py`) or notebooks (`.ipynb`), possibly more than one, covering: data loading, cleaning, validation, feature engineering, strategy/model training or parameter optimization, backtest execution, and final metric/plot generation;
- code shared by the scripts or notebooks (e.g., data loading, features, strategies, backtest, metrics) is kept in separate modules (e.g., in `src/`) and imported, rather than duplicated;
- the repository includes the data and the outputs of every step (e.g., in `data/`): raw data (including data from licensed sources such as Bloomberg), processed data, and final results. The results must be reproducible without downloading any data;
- all key settings are defined in a single configuration file (e.g., `config.yaml`), not scattered across the code;
- random seeds are fixed for all stochastic components, paths are relative to the repository root, and no API keys or credentials are included in the code;
- it includes a way to recreate the environment (e.g., `requirements.txt` or `pyproject.toml` with a lock file such as `uv.lock`, or `environment.yml`) and specifies the Python version;
- it includes a `README.md` with all the information needed to set up the environment, and the commands required to run the code (scripts or notebooks), in order. You can also provide any additional documentation, such as methodology notes or data descriptions (e.g., in `docs/`).

A possible structure:

```
project/
├── README.md
├── pyproject.toml
├── config.yaml
├── 01_download_data.py
├── 02_compute_signals.py
├── 03_compute_metrics.py
├── src/
├── data/
└── docs/
```

Alternatively, a single `main.py` or `main.ipynb` can replace the numbered scripts.

All results must be reproducible starting from the contents of the code repository. All steps will be reproduced by running the code, except for costly or non-deterministic steps (e.g., LLM or paid API calls): their saved outputs will be used as the starting point.

Presentation

The presentation must cover:

- **Introduction:** Your strategy's hypothesis.
- **Data:** What data you used and how you processed it.
- **Methodology:** A detailed explanation of your strategy. This *must* include: *why* you chose your model/AI technique, what parameters you optimized (and how), and the *precise, explicit logic* used to generate the final -1/0/+1 signal (including any thresholds used).
- **Results & Evaluation:** Your performance metrics, including the equity curves.
- **Conclusion:** What worked? What didn't? What are the weaknesses of your system?
- **Contribution statement:** Each member's role.
- **AI tools used**: Which AI assistants you used and for what.

Grading (11 Points Total)

The project is worth **11 points**, broken down as 10 base points and 1 potential bonus point.

The project grade is added to the final exam grade (up to 20 points). Totals above 30 correspond to 30 cum laude.

Base 10 Points:

- **Rigor of Methodology (4 points):** Correct implementation of the framework, avoidance of biases, and a realistic and robust backtest.
- **Strategy Originality & Complexity (3 points):** Quality of the signal generation. A simple SMA crossover will score lower than a well-thought-out ensemble or AI-based system.
- **Performance & Analysis (2 points):** Quality of your performance analysis. Robust out-of-sample risk-adjusted performance will be rewarded (Note: A strategy that performs poorly out-of-sample but is *honestly reported* is better than an overfit one).
- **Code & Presentation Quality (1 point):** Clarity, reproducibility, and professionalism of your deliverables.

Bonus Point

You can earn a bonus point by doing exceptional work that goes significantly beyond the base requirements, **achieving one of these bonus targets:**

- **Market-Neutral Strategy**
  - **Objective:** Design, build, and validate a market-neutral strategy that demonstrates the generation of consistent relevant returns over time, even in periods when the market goes down.
  - **How it is assessed:** OOS beta to a broad market index (e.g., S&P 500) close to zero, low correlation with that index, and positive average returns in the periods when the index has negative returns.

- **Verifiable Superior Risk-Adjusted Performance**
    - **Objective:** Prove that your strategy *conclusively* and *robustly* outperforms its benchmark (e.g., buy-and-hold of the traded assets, or the S&P 500) on a risk-adjusted basis over a sufficiently long and challenging **Out-of-Sample** test period.
    - **How it is assessed:** Over a sufficiently long OOS period, the strategy must show both a higher Sharpe Ratio and a lower Maximum Drawdown than the benchmark. The Sharpe difference should be supported by a statistical test (e.g., bootstrap of the Sharpe difference, or the Deflated Sharpe Ratio).

Use of AI Assistants

You may use AI coding assistants (e.g., Claude, ChatGPT, Copilot) to write code and improve the presentation. Every group member remains fully responsible for the content and must be able to explain every part of the code and methodology.

Project Support & Deadlines

In-Class Project sessions

To support your progress, some lab sessions will be dedicated to projects. Groups will have the opportunity (in turns) to present their progress to the teaching team for direct feedback and to ask specific questions. Groups not currently presenting are expected to use this time to work collaboratively on their project.

Deadlines & Presentation Dates

- **Submission deadline**: November 16th, 23:59.
- **Presentations**: November 17th and November 19th. The teaching team will assign each group to a presentation date.

Grades will be communicated within 2 weeks after the presentations. Late submissions will not be accepted.

Submission Method

A dedicated submission section will be created on the **my.luiss** platform where you will upload your single `.zip` file.